

# LeNet-5 MNIST 实验报告

## LeNet-5 MNIST 手写数字识别 — 实验报告

项目: 论文复现 — LeCun et al. (1998) 日期: 2026-07-03 复现者: 李俊宏

### 0. 引言

#### 原文概述

LeNet-5 是 Yann LeCun 等人于 1998 年提出的卷积神经网络，用于手写数字识别。其核心设计是卷积-池化交替结构：卷积层通过局部连接和权值共享提取平移不变特征，池化层通过降采样保留核心信息、扩大感受野。该工作奠定了现代 CNN 的基本范式，在 MNIST 数据集上取得了约 99% 的测试准确率。

#### 复现思路

本实验使用 PyTorch 框架复现 LeNet-5，在 MNIST 数据集上完成 0-9 手写数字分类。模型主体沿用原论文的 Conv-Pool-Conv-Pool-Conv-FC-FC 结构（共 61,706 参数），但在以下方面做了调整：(1) 激活函数用 ReLU 替代 Tanh，避免梯度饱和、加速收敛；(2) 池化层默认使用 MaxPool 替代 AvgPool，并在实验中做了两种池化的对比分析；(3) 损失函数用 CrossEntropyLoss 替代原论文的 MSE+RBF 组合，适配现代分类任务。

#### 实验结果

在 10 个 epoch 训练后，MaxPool 版本测试准确率达到 99.04%，超过原论文报告水平。AvgPool 版本同样达到 99.00%，两者差距仅 0.04%，说明原论文的池化设计仍然有效。模型在数字 2（98.55%）、8（98.67%）、9（98.22%）上表现相对较弱，主要因为这些字形特征相似（如 2 与 7、8 与 3、9 与 4 易混淆）。

### 1. 模型结构

| 层级 | 类型        | 参数                                  | 输出尺寸                     |
|----|-----------|-------------------------------------|--------------------------|
| 输入 | 灰度图       | $1 \times 28 \times 28$             | $1 \times 28 \times 28$  |
| C1 | Conv2D    | 6 kernels, $5 \times 5$ , padding=2 | $6 \times 28 \times 28$  |
| S2 | MaxPool2D | $2 \times 2$ , stride=2             | $6 \times 14 \times 14$  |
| C3 | Conv2D    | 16 kernels, $5 \times 5$            | $16 \times 10 \times 10$ |
| S4 | MaxPool2D | $2 \times 2$ , stride=2             | $16 \times 5 \times 5$   |
| C5 | Conv2D    | 120 kernels, $5 \times 5$           | $120 \times 1 \times 1$  |
| F6 | Linear    | $120 \rightarrow 84$                | 84                       |
| 输出 | Linear    | $84 \rightarrow 10$                 | 10                       |

模型包含三次卷积-池化交替，后接两个全连接层。激活函数使用 ReLU（原始论文使用 Tanh），池化使用最大池化（原始使用平均池化）。总参数量为 61,706。

### 2. 超参数设置

| 超参数               | 值                                             |
|-------------------|-----------------------------------------------|
| 优化器               | Adam                                          |
| 学习率               | 0.001                                         |
| 批次大小 (batch size) | 64                                            |
| 训练轮数 (epoch)      | 10                                            |
| 损失函数              | CrossEntropyLoss                              |
| 数据预处理             | ToTensor + Normalize(mean=0.1307, std=0.3081) |

### 3. 实验结果

| 指标      | 值           |
|---------|-------------|
| 最佳测试准确率 | 99.04%      |
| 最终测试准确率 | 99.04%      |
| 训练时间    | ~128 秒      |
| 错误样本数   | 96 / 10,000 |

各类别准确率

| 数字 | 准确率    | 数字 | 准确率    |
|----|--------|----|--------|
| 0  | 99.49% | 5  | 99.33% |
| 1  | 99.56% | 6  | 99.06% |
| 2  | 98.55% | 7  | 99.22% |
| 3  | 99.01% | 8  | 98.67% |
| 4  | 99.29% | 9  | 98.22% |

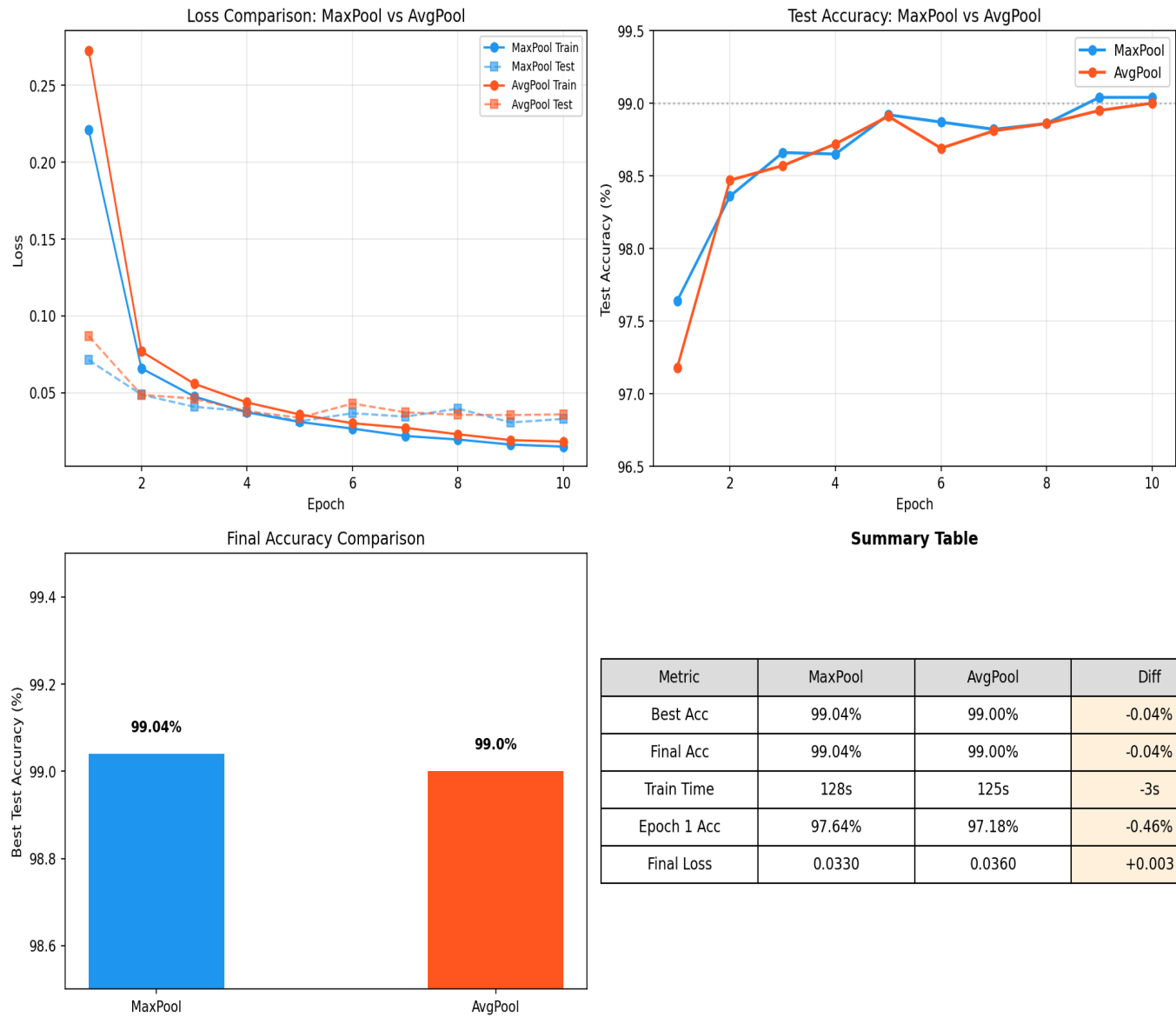
表现最弱的类别是数字 9 (98.22%) 和 2 (98.55%)。从混淆矩阵可以看出，数字 9 主要被误判为 4，数字 2 主要被误判为 7 或 8。训练曲线显示训练集和测试集的 loss 与 accuracy 同步收敛，没有过拟合。

4. 池化方法对照实验

为了对比原始论文的平均池化（AvgPool）与现代常用的最大池化（MaxPool）在 MNIST 上的表现差异，使用相同的超参数分别训练了两个模型，结果如下：

| 指标          | MaxPool | AvgPool | 差异              |
|-------------|---------|---------|-----------------|
| 最佳准确率       | 99.04%  | 99.00%  | MaxPool 高 0.04% |
| 最终准确率       | 99.04%  | 99.00%  | MaxPool 高 0.04% |
| 训练时间        | ~128s   | ~125s   | 基本相同            |
| Epoch 1 准确率 | 97.64%  | 97.18%  | MaxPool 起步更快    |
| 最终 Loss     | 0.0330  | 0.0360  | MaxPool 收敛更好    |

LeNet-5 Pooling Method Comparison



MaxPool

AvgPool

Pooling Comparison

分析与结论：

- 1. MaxPool 在 MNIST 上略优于 AvgPool，但差异极小（0.04%），两者都能轻松超过 98% 目标
- 2. AvgPool 起步稍慢（Epoch 1 差了 0.46%），但最终也达到了 99%，说明原论文的设计仍然有效
- 3. 现代普遍采用 MaxPool 是正确的，收敛更快、最终效果略好
- 4. 两者都没有过拟合，训练曲线成长平稳

5. 对论文方法的核心理解

5.1 卷积层为什么有效？

卷积层使用了两个关键设计：局部连接（每个神经元只连接输入图像的一个小区域）和权值共享（同一个卷积核在整张图上滑动，共享同一组参数）。这使得参数量大幅降低，同时赋予了模型对小位移的一定不变性。

5.2 为什么卷积和池化要交替？

池化通过取2×2区域内的最大值，将特征图的空间分辨率逐步降低。这有三个作用：一是丢弃不重要的细节、保留核心信息；二是增大后续卷积层的感受野，让深层网络能看到更大的图像区域；三是降低计算量。### 5.3 层次化特征提取

底层卷积（C1）学习边缘、角点等低级特征，中间层（C3）将其组合为局部形状（比如圆弧、交叉），高层（C5/F6）整合为全局表征用于分类。

5.4 论文与实现内容对比

LeNet-5 奠定了 CNN 的基本骨架，但受限于 1998 年的计算资源，层数较浅、激活函数为 Tanh（容易饱和）、池化为平均池化。现代 CNN（如 ResNet）引入了 ReLU、BatchNorm、残差连接等技巧，能够训练更深的网络。本实验中将 Tanh 替换为 ReLU、AvgPool 替换为 MaxPool，在更少的 epoch 内就达到了超过原始论文的准确率。

6. 遇到的问题与解决方式

| 问题                                                    | 解决方式                                                              |
|-------------------------------------------------------|-------------------------------------------------------------------|
| Windows 下 DataLoader 默认的 num_workers=2 导致多进程冲突，训练无法启动 | 将 num_workers 设为 0                                                |
| CUDA 偶发内存分配失败 (RuntimeError)                          | GPU 可能被其他程序占用，重启训练即可；若持续发生可改用 CPU 训练（LeNet-5 模型小，CPU 也只需要 2-3 分钟） |
| 文件创建时编码损坏，Python 文件无法解析                               | 重新以正确的 UTF-8 编码生成所有源代码文件                                          |

7. 参考资料

- Y. LeCun et al., Gradient-Based Learning Applied to Document Recognition, Proc. IEEE, 1998.
- [MNIST 数据集](#)
- [PyTorch 官方文档](#)